

Customer Churn Prediction Model

Development Report

Executive Summary

This report documents the development of a machine learning model to predict customer churn risk. The project followed a structured approach to build, train, and optimize various classification models using a dataset of approximately 36,000 customers with 20 features. Through rigorous evaluation and optimization, we identified Logistic Regression as the most effective model for predicting customer churn, achieving an F1-score of 0.6786 and a ROC-AUC of 0.9058 on the test set. These robust performance metrics will enable proactive customer retention strategies.

1. Introduction

Customer churn prediction is critical for businesses to identify at-risk customers and implement targeted retention strategies. This project aimed to develop an accurate and reliable machine learning model that can classify customers based on their churn risk. The model development process followed industry best practices and a methodical approach to ensure high-quality results.

2. Data Preparation

2.1 Data Overview

- Dataset size: ~36,000 samples with 20 features
 - Target variable: churn_risk_score (multi-class classification with 5 levels)
 - The data was previously cleaned and processed through established pipelines
-

2.2 Preprocessing Pipeline

The data preparation process utilized two existing pipelines:

- cleaning_pipeline: Handled missing values, outliers, and other data cleaning tasks
 - featureEng_encoder_scaler_pipeline: Performed feature engineering, encoding of categorical variables, and feature scaling
-

2.3 Train-Test Split

- The dataset was split into training (80%) and testing (20%) sets
 - Stratified sampling was employed to ensure balanced class distribution in both sets
 - Random seed was set to 42 for reproducibility
-

2.4 Class Imbalance Handling

- Class distribution was analyzed to identify potential imbalance
 - SMOTE (Synthetic Minority Over-sampling Technique) was applied to balance the training data when the minority class representation fell below 30%
 - This ensured models were not biased toward the majority class during training
-

3. Model Selection

Multiple classification models were selected for evaluation, each with different strengths:

3.1 Base Models

1. Logistic Regression

- Served as a baseline model
- Provided good interpretability and model transparency
- Relatively fast training and prediction

2. Decision Tree

- Offered clear decision boundaries and feature importance
- Captured non-linear relationships in the data
- Potentially prone to overfitting

3. Random Forest

- Ensemble of decision trees to reduce overfitting
- Robust to outliers and noise
- Provided feature importance measures

4. Gradient Boosting

- Sequential ensemble method focusing on errors
- Often delivers high performance for structured data
- Requires careful tuning to avoid overfitting

5. XGBoost

- Optimized implementation of gradient boosting
 - Known for state-of-the-art performance in tabular data
 - Efficiently handles missing values
-

4. Model Training Methodology

4.1 Cross-Validation Strategy

- 3-fold stratified cross-validation was implemented for all models
 - This approach ensured robust performance estimation by testing on multiple data subsets
 - Each model was trained and evaluated three times on different data partitions
-

4.2 Training Process

For each model:

- Cross-validation was performed to estimate generalization performance
 - Multiple metrics were calculated: accuracy, precision, recall, F1-score
 - Model was then trained on the entire training set
 - Training time was recorded for efficiency comparison
-

4.3 Evaluation Metrics

The following metrics were used to comprehensively evaluate model performance:

- Accuracy: Overall correct prediction rate
 - Precision: Proportion of true positives among positive predictions
 - Recall: Proportion of actual positives correctly identified
 - F1-Score: Harmonic mean of precision and recall
 - ROC-AUC: Area under the Receiver Operating Characteristic curve (One-vs-Rest)
-

5. Model Evaluation Results

5.1 Base Models Performance

Each model was evaluated on the held-out test set, with detailed metrics collected for comparison. The results are summarized below:

Model	CV Accuracy	Test Accuracy	Precision	Recall	F1- Score	ROC- AUC (OVR)	Training Time (s)
Logistic Regression	0.6864	0.6864	0.6847	0.6864	0.6673	0.9030	15.81
Decision Tree	0.6156	0.6156	0.6132	0.6156	0.6143	0.7426	10.96
Random Forest	0.6791	0.6791	0.6702	0.6791	0.6496	0.8954	22.26
Gradient Boosting	0.6862	0.6862	0.6802	0.6862	0.6665	0.9023	548.37
XGBoost	0.6893	0.6893	0.6829	0.6893	0.6673	0.9056	25.98

5.2 Performance Analysis

- Logistic Regression and XGBoost outperformed other models in terms of F1-score (0.6673) and ROC-AUC (0.9030 and 0.9056, respectively).
- Decision Tree had the lowest performance, with a test accuracy of 0.6156 and F1-score of 0.6143, likely due to overfitting.

- Random Forest showed balanced performance but underperformed in F1-score (0.6496) compared to Logistic Regression and XGBoost.
 - Gradient Boosting achieved similar performance to Logistic Regression but required significantly longer training time (548.37 seconds).
 - Visual analysis included confusion matrices, ROC curves, and Precision-Recall curves for each model to assess performance across thresholds.
 - The best-performing model was selected based primarily on F1-score, with Logistic Regression chosen due to its interpretability and efficiency.
-

6. Hyperparameter Tuning

6.1 Tuning Methodology

- RandomizedSearchCV was employed for efficient hyperparameter optimization
 - 10 parameter combinations were sampled for Logistic Regression
 - 3-fold cross-validation assessed each parameter set
 - F1-weighted was used as the optimization metric
-

6.2 Parameter Grids

The parameter grid for Logistic Regression was:

- C: [0.001, 0.01, 0.1, 1, 10, 100]
 - penalty: ['l1', 'l2', 'elasticnet', None]
 - solver: ['newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga']
-

6.3 Tuning Results

- Best parameters: {'solver': 'saga', 'penalty': 'l1', 'C': 0.1}

- Best cross-validated F1-score: 0.7027
 - Tuning time: 218.32 seconds
 - The tuned Logistic Regression model improved performance over the base model:
 - Test Accuracy: 0.6991 (vs. 0.6864)
 - Precision: 0.7015 (vs. 0.6847)
 - Recall: 0.6991 (vs. 0.6864)
 - F1-Score: 0.6786 (vs. 0.6673)
 - ROC-AUC: 0.9058 (vs. 0.9030)
-

7. Feature Importance Analysis

7.1 Feature Ranking

- Feature importance was extracted from the tuned Logistic Regression model using coefficients.
 - Features were ranked by their contribution to model predictions.
 - Top 20 most influential features were identified and visualized (not detailed in results provided).
-

7.2 Insights

- Key drivers of churn were identified based on feature importance.
 - This information provides valuable business insights for designing targeted retention strategies.
 - The analysis helps focus future data collection efforts on the most predictive attributes.
-

8. Final Model Development

8.1 Model Selection Criteria

- The final model was selected based on comprehensive performance metrics.
 - F1-score was prioritized as it balances precision and recall.
 - Consideration was given to model interpretability, training time, and deployment simplicity.
-

8.2 Final Model

- The tuned Logistic Regression model was selected as the final model.
 - Performance Metrics:
 - Accuracy: 0.6991
 - Precision: 0.7015
 - Recall: 0.6991
 - F1-Score: 0.6786
 - ROC-AUC: 0.9058
 - Both the standalone model and the complete preprocessing pipeline were saved for deployment.
-

9. Deployment Preparation

9.1 Model Persistence

- The final trained model was saved as `best_churn_model_logistic_regression.pkl` using pickle serialization.

- The complete pipeline (preprocessing + model) was saved as `complete_churn_pipeline.pkl` to simplify deployment.
 - This approach ensures consistent preprocessing in production.
-

9.2 Prediction Function

- A utility function `predict_churn()` was created for easy model implementation.
 - The function handles both class predictions and probability scores.
 - It encapsulates the entire preprocessing and prediction workflow.
-

10. Model Evaluation Report

10.1 Report Structure

1. A comprehensive evaluation report was generated containing:
2. Model name and type: Logistic Regression
3. Optimized parameters: `{'solver': 'saga', 'penalty': 'l1', 'C': 0.1}`
4. Performance metrics on test data:
5. Accuracy: 0.6991
6. Precision: 0.7015
7. Recall: 0.6991
8. F1-Score: 0.6786
9. ROC Essential elements of the model evaluation report:
10. Cross-validation methodology and results: 3-fold stratified cross-validation, best CV F1-score of 0.7027
11. Training information: Training set size (33,355 samples), test set size (5,733 samples), SMOTE applied
12. Class balancing techniques: SMOTE

13. The report was saved as model_evaluation_report.pkl.

10.2 Performance Summary

The final model achieved:

- High accuracy (0.6991) in predicting customer churn across five risk levels.
 - Balanced precision (0.7015) and recall (0.6991).
 - Strong ROC-AUC score (0.9058), indicating excellent discrimination ability.
 - Reliable performance across different customer segments, particularly for high-risk classes (e.g., class 5.0 with F1-score of 0.78).
-

11. Conclusion and Recommendations

11.1 Project Achievements

- Successfully developed a robust churn prediction model using Logistic Regression.
 - Implemented a comprehensive evaluation strategy with 3-fold cross-validation.
 - Optimized model parameters, improving F1-score from 0.6673 to 0.6786.
 - Created deployment-ready artifacts, including model and pipeline files.
-

11.2 Future Improvements

Potential enhancements for future iterations:

- Explore more advanced ensemble techniques, such as stacking Logistic Regression with XGBoost.
- Implement neural network approaches for comparison.

- Conduct more extensive feature engineering to uncover additional predictive attributes.
- Develop a model monitoring system for performance drift detection.

11.3 Business Applications

The churn prediction model enables:

- Proactive identification of at-risk customers.
 - Targeted retention campaigns based on churn risk levels.
 - Resource allocation optimization for customer retention efforts.
 - Customer lifetime value enhancement through timely interventions.
-

Appendix: Implementation Details

A1. Development Environment

- Python: 3.x
 - Libraries: scikit-learn, pandas, numpy, matplotlib, seaborn, imblearn, xgboost
 - Execution platform: Google Colab
-

A2. Execution Time

- Total execution time: Approximately 20-35 minutes on Google Colab
 - Most time-consuming step: Hyperparameter tuning (~218.32 seconds for Logistic Regression)
-

A3. Code Structure

The implementation followed a modular approach with functions for:

- Model training and cross-validation
 - Performance evaluation and visualization
 - Hyperparameter tuning
 - Report generation
 - Prediction utility
-

Finally, This structured development process ensured a reliable, high-performing churn prediction model that meets the project objectives and delivers business value.